

WookongDB 查询重写技术研究

徐言诗, 张心怡, 卢 卫, 杜小勇⁺

中国人民大学, 信息学院, 北京 100872

+ 通信作者 E-mail: duyong@ruc.edu.cn

摘 要:为应对云原生分层架构下数据库查询优化性能不佳的挑战,提出基于大语言模型(LLM)的智能查询重写方法。传统基于硬编码规则的查询重写存在覆盖率低、扩展性差等问题,而直接应用大语言模型则面临幻觉生成与执行有效性验证不足的缺陷。通过融合规则规范、历史案例与元数据构建动态知识库,建立包含语法-语义-形式化验证、候选重写评估与自修正反馈的三阶段校验机制,有效控制模型生成内容的可靠性。通过云原生架构感知模块与混合负载适配机制,系统在分布式环境下保持优化稳定性。在实现路径上,采用查询分类机制区分基础查询与复杂查询:基础查询沿用数据库原生优化器,复杂查询激活大语言模型处理流程,通过混合检索策略动态获取优化知识。实验验证表明,在TPC-H基准测试中,相较于WookongDB原生优化器,该方法能实现平均50.3%的查询效率提升。在DSB混合负载测试中,重写有效性达到96.2%,较直接LLM方案提升35.0个百分点。该研究不仅为云原生数据库的查询优化提供了新思路,研究中构建的智能重写系统也已在WookongDB生产环境中验证了实用价值,对推动数据库智能化发展具有重要理论意义和工程价值。

关键词:云数据库;查询重写;大语言模型;检索增强生成

文献标志码:A **中图分类号:**TP311

Research on Query Rewriting Technology in WookongDB

XU Yanshi, ZHANG Xinyi, LU Wei, DU Xiaoyong⁺

School of Information, Renmin University of China, Beijing 100872, China

Abstract: To address the challenges of suboptimal query optimization in cloud-native layered architectures, this paper proposes an intelligent query rewriting method powered by large language models (LLMs). Traditional rule-based query rewriting suffers from low coverage and poor scalability, while direct application of LLMs faces issues such as hallucination generation and insufficient execution validity verification. This paper constructs a dynamic knowledge base by integrating rule specifications, historical cases, and metadata. A three-stage verification mechanism is established, encompassing syntax-semantics-formal validation, candidate rewriting evaluation, and self-correcting feedback, to ensure the reliability of model-generated content. A cloud-native architecture-aware module and hybrid workload adaptation mechanism are introduced to maintain optimization stability in distributed environments. Implementation-wise, a query classification mechanism distinguishes between basic queries (optimized by the native database optimizer) and complex queries (processed via LLMs). A hybrid retrieval strategy dynamically fetches optimization knowledge. Experimental results on the TPC-H benchmark show that, compared with the native optimizer of WookongDB, the proposed method achieves an average 50.3% query efficiency improvement. In DSB mixed-workload tests, rewriting effectiveness reaches 96.2%, outperforming direct LLM-based approaches by 35.0 percentage points. This research not only provides a novel approach for query optimization in cloud-native databases but also demonstrates the practical value of the developed intelligent rewriting system in

基金项目:国家自然科学基金(62441230, 62472429, 62461146205)。

This work was supported by the National Natural Science Foundation of China (62441230, 62472429, 62461146205).

收稿日期:2025-07-29 **修回日期:**2025-09-23

production environment of WookongDB. It holds significant theoretical and engineering implications for advancing intelligent database systems.

Key words: cloud databases; query rewriting; large language models; retrieval-augmented generation

数据库作为新型生产要素,是新质生产力发展的重要动力。随着国家数据局的成立和全球人工智能浪潮的推动,数据库技术和产品不断创新,是支撑数据存储与计算的关键载体^[1-2]。数据库产业持续蓬勃发展,技术创新和行业应用将更加深入,推动中国向“数据强国”迈进。

随着大数据与云原生技术的快速发展,数据库系统面临两大重要挑战:混合负载处理能力与智能优化需求。数据库作为信息存储、管理以及处理的核心工具,其性能和效率对各行业的数据驱动决策至关重要。然而,传统查询优化器在云环境下暴露出显著局限性。一方面是基于固定且有限规则的传统重写策略难以适应极其复杂的混合负载及行列混存的存储结构,另一方面是云原生架构数据库的存储-计算-内存多层分离特性^[3-9]导致查询重写所需信息复杂化,传统的启发式策略难以应对。

与此同时,大语言模型与检索增强生成(retrieval-augmented generation, RAG)技术的突破为数据库优化提供了新思路。二者在多种数据库任务中展现出了巨大潜能^[10-11],包括查询重写^[12-15]、索引调优^[15]、数据库诊断^[16]等,形成了LLM4DB这一新兴交叉学科方向。研究表明,LLM(large language model)可通过语义理解自动生成等价查询变体,而RAG技术能有效整合数据库系统查询重写所需的信息,缓解大语言模型的“幻觉”问题。这为构建自适应查询重写系统提供了理论可能,但其在云原生环境下的应用仍存在一些空白,包括架构适配缺失、优化维度单一以及领域知识利用不足等^[17-18]。

本研究重点聚焦于一款云数据库 WookongDB,其存储-计算-状态三层分离架构为查询优化提出了新要求。在 WookongDB 这类现代数据库管理系统中,查询优化是确保高效数据检索和处理的核心环节。它通过一系列复杂的分析、转换和选择适当的执行策略,旨在将用户提交的SQL(structured query language)查询转化为高效的物理执行计划,从而最大限度地提高性能并降低资源消耗(如CPU、内存、磁盘I/O等)。查询重写是查询优化流程中的首步,优质的查询重写对后续流程至关重要。

本研究提出智能查询重写技术体系,通过大语言模型与领域知识的深度耦合,构建SQL查询智能重写系统,突破传统查询重写方法的局限性。本研究的贡献如下:

(1)提出了基于大语言模型和检索增强生成技术的语义保持SQL查询重写框架。该框架通过SQL查询分类与云原生架构感知等技术,有效支持了分层云原生架构下的重写需求。

(2)针对大模型带来的幻觉问题,提出了多源知识引导的混合检索增强技术,建立了包括重写规则、模式知识、历史重写结果、运行时监控以及云环境特殊知识等的知识库,构建从语法-语义-形式化验证到执行反馈的闭环优化机制等。

(3)在云原生数据库 WookongDB 上开发实现了查询重写模块 Wookong-QR,实验结果表明,相比于传统的查询重写方法,在重写后查询的执行性能上有明显提高。

1 查询重写任务

查询重写任务本质上是从一条SQL查询到另一条等价查询的转换过程。该过程旨在通过对原始查询进行优化,以提升数据库系统的性能和效率。查询重写的底限要求是保持查询的等价性,即重写前后的SQL查询必须做到执行结果完全一致。而查询重写的核心目标是提高SQL语句的执行性能,需要综合考虑多种因素来提升性能,如数据表模式、数据分布、访问模式及系统资源等。

传统基于规则的查询重写方法是给定一条SQL查询表示为 q ,一组规则表示为 $r_1, r_2, \dots, r_n$,规则序列中的每一条规则都是对 q 的一个等价变换,作用在 q 上的这组规则序列即为查询重写操作。

定义1(查询重写规则) 规则可以形式化地定义为一个三元组,如下所示。它包括转换 T ,规则应用条件 C 与规则匹配函数 F ,转换与规则应用条件一般由自然语言描述,而规则匹配函数由代码直接实现。转换 T 包括两部分,原查询 q 与新查询 q^* 的模板。

$$r = \langle T, C, F \rangle$$

应用规则的顺序很重要,不同顺序可能会产生不同重写结果。查询重写完成后,最终生成一条新查询 q^* ,它与原SQL查询不同但等价。这个过程表示为:

$$q^* = R(q)$$

其中, q^* 为新查询, q 为原始SQL查询,规则序列 $R = [r_1, r_2, \dots, r_n]$ 表示 n 条重写规则及其应用顺序。表1是一个基于规则的SQL查询重写示例。

表1 基于规则的SQL查询重写示例

Table 1 Example of rule-based SQL query rewriting

描述	内容示例
原始查询 q	SELECT emp_id, emp_name FROM employees WHERE emp_id IN (SELECT emp_id FROM salaries WHERE salary > 50000);
新查询 q^*	SELECT e.emp_id, e.emp_name FROM employees e JOIN salaries s ON e.emp_id = s.emp_id WHERE s.salary > 50000;
规则序列 R	["SUB_QUERY_REMOVE"]

传统的开源关系型数据库如MySQL等,均为此类基于规则的查询重写。在WookongDB这类云原生复杂负载环境下,本研究引入大语言模型与检索增强生成等技术,聚焦于核心组件,定义了一个新的查询重写函数 F_{QR} :

$$q^* = F_{QR}(q, L, P, S, T, K)$$

其中, q^* 表示优化后的SQL查询; L 表示大语言模型,用于生成新的查询和规则序列; P 为规则决策池,包含当前可用的重写规则,初始化时可以使用Apache Calcite中的一些常用规则。 S 表示相似查询列表,辅助查询重写过程,包含若干个元组 $\langle q, q^*, R_s, cost_s \rangle$; T 是SQL等效性校验器与候选重写评估器,确保新生成的查询与原查询语义等价且相比原查询效果更优; K 为辅助查询重写的知识库,包含数据表模式等信息。

2 基于大语言模型的查询重写框架

针对SQL查询重写任务,本文通过大语言模型强大的文本生成能力与自我纠正功能来主导查询重写,并采用检索增强生成(RAG)技术导入相关信息来辅助重写,结合多种技术提出了基于大语言模型的新型查询重

写框架Wookong-QR,后文简称框架。框架整体上由两个核心模块组成,分别是知识库增强模块与新查询生成模块,如图1所示。

2.1 重写预备模块

重写预备模块主要是对SQL查询进行预处理。开始重写时,每次处理一条SQL语句,基于此查询生成一条含有SQL语句的基础重写提示,最后将合成的提示导出到知识库增强模块中进行下步操作。

2.1.1 SQL查询分类

SQL查询分类模块构建基于机器学习的查询分类器,对输入SQL查询实施复杂度评估与任务路由。通过特征工程提取查询的语法结构特征与执行历史特征,结合动态阈值策略将查询划分为基础型与复杂型两类。基础型查询直接路由至数据库原有的传统优化器处理,复杂型查询则触发大语言模型处理流水线。同时基于提示工程生成上下文感知的提示模板,给后续的新查询生成模块中提供核心输入。

2.1.2 云原生架构感知的提示词生成

该过程基于提示工程原理,通过结构化模板和SQL语句结合,构建基础提示 p_0 ,该过程遵循分层提示原则。基础提示模板包含以下核心提示:

$$p_0 = \langle R, T, C \rangle$$

其中, R 为角色定义(SQL重写优化专家), T 是任务描述(把初始SQL查询重写为等价高效查询), C 则是约束条件(基于给定知识,仅输出最终SQL)。

但这并非最终给大语言模型输入的提示,在知识库增强模块中,还需要用RAG技术注入更多信息帮助SQL查询精确重写。为支持后续知识增强,在提示中设置了Assistant部分,给参考信息预留知识插槽,增强后提示如下:

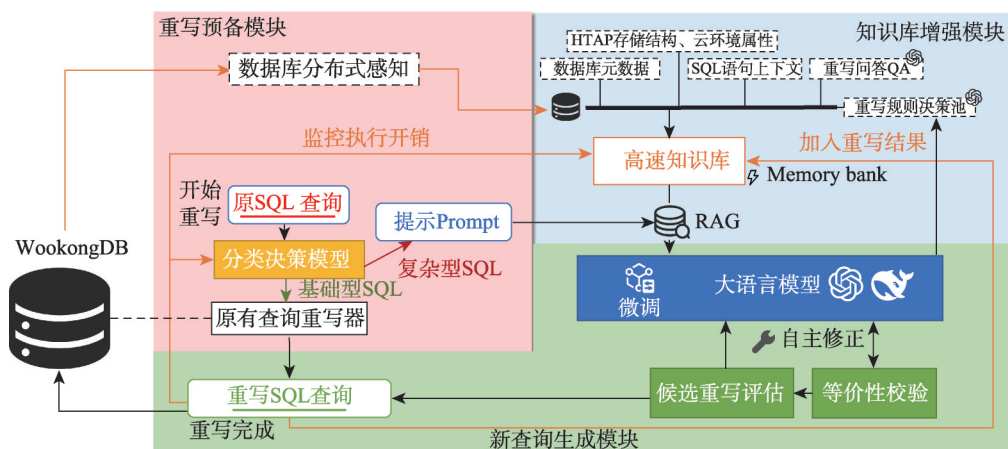


图1 基于大语言模型的查询重写框架Wookong-QR

Fig.1 Wookong-QR: query rewriting framework based on large language models

$$p_{\text{enhanced}} = p_0 \oplus \bigoplus_{k=1}^K \phi(k)$$

其中, $\phi(k)$ 表示从知识库 K 检索的第 k 个相关知识片段, 由知识增强模块动态填充。 $\oplus$ 表示知识片段拼接操作。当前阶段, 初始化 $K=1$, $\phi(1)=\emptyset$ 。

本部分引入的一些主要信息如下:

模式知识: 存储数据库的静态元信息, 包括表结构定义、索引配置及约束条件。例如对用户查询中涉及的“salaries”表, 检索其包含的列名、主键约束以及建立的 B+ 树索引信息。除了这些传统数据库中具有的信息, 还需要考虑 WookongDB 的行列混存结构, 以及“存储-计算-内存”分层云数据库的数据分布特性。

运行时监控: 实时采集数据库负载特征, 包括 CPU 利用率、内存带宽占用、存储 I/O 吞吐量、云环境下的网络传输代价等多项指标, 构建多维性能画像。

除了这些信息外, WookongDB 这类分离架构的云数据库具有物理资源解耦、弹性扩展、存储层数据共享等特征。在分离架构下, 传统基于本地磁盘 I/O 的一些优化假设失效, 网络延迟与带宽成为新的性能瓶颈。这些新特性会影响查询重写的效果, 因此在知识库中引入一些云原生架构下的信息, 用来辅助后续模块进行高效的查询重写。

需要考虑的云环境特殊知识有分布式感知查询重写、存储层计算能力的利用、混合负载存储感知的查询重写、缓存感知的重写、复杂查询的分解与并行化等。

知识融合过程可形式化为:

$$\mathcal{K}^{(i)} = \mathcal{F}(\mathcal{K}_s, \mathcal{K}_m^{(i-1)}, \mathcal{K}_c)$$

其中包括了各类知识源:

(1) $\mathcal{K}_s$: 模式知识, 包含表结构 T 、索引 I 及约束 C 等数据库信息, 也包括 WookongDB 特有的行列混存结构 S , 以及数据分布特性 D 等。

(2) $\mathcal{K}_m$: 运行时监控数据, 记录 CPU 利用 τ_{cpu} 、I/O 耗时 τ_{io} 等多项指标。

(3) $\mathcal{K}_c$: 云环境下的特殊知识。

2.2 知识库增强模块

知识库增强模块核心目标是通过多个数据源提升大语言模型的领域适应性。本模块采用检索增强生成 (RAG) 技术构建多模态知识库, 集成数据库模式描述、历史重写案例等多源信息为查询重写提供辅助。大语言模型在处理查询重写这类特定领域知识时, 往往面临“记忆”限制, 导致生成内容在准确性、相关性和时效性上存在不足, 需要 RAG 技术补足这方面的短板。框架充分利用互联网信息, 从多个渠道获取重写规则等知识初始化知识库, 让知识库热启动, 提升查询重写效果。

除了重写规则外, 这里还用到了历史重写结果, 即系统会记录过去 30 天内所有成功优化的查询案例, 每个案例包含原始查询、重写查询、重写规则序列及性能指标, 形成决策四元组 $\langle q, q^*, R, \text{cost} \rangle$ 。通过时间衰减因子加权处理, 确保近期案例具有更高参考价值。

传统基于规则的查询重写方法受限于静态规则库的覆盖范围与人工维护成本, 小模型方法存在数据依赖且现有固定规则数量有限^[17, 19], 均难以应对云原生环境下混合负载的动态优化需求。针对这一挑战, 本文提出基于大语言模型的动态规则生成与演化技术, 其核心流程如下:

(1) 规则挖掘。通过大语言模型的深度语义解析能力解构查询逻辑特征。给定输入查询 q , 首先提取其抽象语法树 (abstract syntax tree, AST) 的拓扑结构 T_q 与操作符序列 O_q , 并使用余弦距离算法从多源知识库 K 中匹配候选规则集 R_c 。形式化表示如下:

$$R_c = \text{Retrieve}(\phi(T_q) \oplus \psi(O_q), K)$$

其中, $\phi(\cdot)$ 为 AST 编码器, $\psi(\cdot)$ 为面向操作符的语义嵌入函数, $\oplus$ 表示特征拼接操作。

具体来说, 对于同一个查询, 可能会有多种不同的规则序列能完成查询重写操作, 尝试从中选取到最合适的规则。知识检索的过程采用匹配-过滤策略: 首先, 计算当前查询的向量化表征 $\phi(T_q) \oplus \psi(O_q)$, 使用余弦距离算法计算当前查询和历史查询集 $\{q_i\}_{i=1}^n$ 的相似性, 召回历史查询对应的重写规则等知识; 然后, 利用指标过滤器 Filter 过滤掉不匹配当前系统状态的结果, 并通过 cost 对候选结果进行排序, 选出更优的结果。

该知识检索策略形式化表示如下:

$$\text{Retrieve}(q) = \text{Cosine}(q, \{q_i\}_{i=1}^n) \cap \text{Filter}(\mathcal{K}_m, \{\text{cost}\})$$

(2) 规则合成。设计轻量级规则合成引擎实现知识沉淀。对于大语言模型生成的新颖优化模式 $\mathcal{P}_n$ (如负载感知的谓词下推策略), 通过以下四个步骤将其转化为可复用规则:

①规则规范化: 将自然语言描述的模式转换为规范化的标准形式规则。

②效果验证: 在验证集 $\mathcal{D}_v$ 上检验规则的有效性与安全性。

③冲突检测: 分析解决与现有规则库的潜在冲突。

④动态注入: 将通过检验的规则增量更新至规则池 P 。

2.3 新查询生成模块

新查询生成模块主要是利用大语言模型来生成新查询。本模块接受重写预备与知识库增强两个模块的

输入,即 p_{enhanced} 。大语言模型生成候选重写,接下来进行语法-语义-形式化验证,判断重写的结果和原始查询是否等价,保证查询的正确性。随后,进入候选重写评估子模块,判断该重写结果是否更优。如果出现前后查询不等价或者重写效果不佳的状况,表示重写过程出现问题。触发基于强化学习的自主修正流程,将无效低效重写结果反馈至修正管道,通过多轮迭代优化重新生成结果。最终,生成语法合规的重写查询。

2.3.1 语法-语义-形式化验证

语法-语义-形式化验证子模块的核心目标是判断大语言模型生成的候选重写和原始查询是否等价,这是一个较为复杂但又至关重要的环节。尤其对于DSB基准这类复杂的查询而言,很难做到等价和无错误重写,设置语法-语义-形式化验证模块很有必要。为了判断重写后的查询和原始查询是否等价,本研究设置了语法相似性检测、语义测试与完全形式化验证三个阶段,确保新查询与原查询在语法和语义上的等价性。三个阶段层层递进,全部通过才说明查询等价。

(1)语法相似性检测。首先需要确定查询是否符合SQL语法规则。这里可以使用EXPLAIN语句来尝试执行,保障SQL查询的可执行性。确认语法正确能够执行后,再使用SQL解析器如ANTLR来生成解析树,进行结构比对。通过比较原始查询和重写后查询的抽象语法树,确保两者在结构上的一致性,防止因不正确的语法转换导致的错误。定义语法相似度:

$$\text{Sim}_{\text{syn}}(q, q') = \frac{|T(q) \cap T(q')|}{|T(q) \cup T(q')|}$$

其中, $T(\cdot)$ 表示查询的语法标记集合。可以取阈值 $\theta_{\text{syn}} = 0.9$, 当相似度低于阈值时认为语法上不等价,该项测试不通过。反之如果测试通过,则进入下一个阶段语义测试。

(2)基于语义的查询等价性测试。本文利用SlabCity^[20], 一个基于语义的查询等价性测试工具进行较低成本的等价性测试。该测试的核心思路是判断能否找出一个满足完整性约束 ϕ 的模拟数据库 D , 使得原查询与新查询结果不同,若能找到则说明语义不等价,反之则语义等价。然而,随机生成模拟数据库 D 的方法成本过高不太可行,需要用合理的方法高效地模拟所需数据。因此,SlabCity对原查询 q 与新查询 q^* 的句法结构进行分析,提取出一些最能暴露差异的要素,如表2所示。

针对发现的这些敏感要素,将各个要素用数学语言进行描述,实现自动化的模拟测试。具体来说,挖掘出的查询敏感要素,被编码为逻辑公式 ϕ_1 , 原有完整性约束编码为公式 ϕ 。通过约束求解器Microsoft Z3求解

表2 语义测试核心要素举例

Table 2 Examples of core elements of semantic testing

要素	测试数据生成
筛选条件	比如 WHERE age>20, 说明这个查询对年龄敏感。生成多组满足或不满足该 WHERE 条件的测试用例, 例如 age=19/20/21 的情况, 测试重写后的查询是否错误地放宽或收紧了条件
连接方式	比如 JOIN user ON id, 说明对 ID 匹配敏感, 收集 JOIN 条件以生成不会产生空中间连接结果的输入
特殊操作	DISTINCT、GROUP BY 等。例如, DISTINCT 说明对重复数据敏感, 可以生成含有几组重复数据的数据库, 测试 DISTINCT 是否真的生效, 避免错误去重

$\phi \cap \phi_1$ 不为空的满足赋值, 每个赋值即对应一个数据库输入 D 。若 ϕ 与 ϕ_1 矛盾无解, 则表明无法生成有效测试输入, 此时需转向后续验证方法。若原查询与新查询在 D 上执行结果不同, 则未通过等价性测试; 反之转向下一阶段进行完全形式化验证。

(3)完全形式化验证。本阶段整合了COSETTE完全验证器,能够证明查询在所有可能输入上的等价性^[20]。完全形式化验证可以提供最高保证,但其代价更高,本研究中仅在前两个环节无法证伪时才谨慎使用完全验证。在实现过程中采用了模块化设计,支持集成更多验证工具,实现组件替换升级。如果等效性校验通过,候选查询进入下一个子模块,即候选重写评估。

2.3.2 候选重写评估

候选重写评估模块的目的是评估重写后的查询有没有比原查询更优,需要预测或实测前后查询的执行效率,防止负面优化。前文提到,本框架可以监控查询的实际执行过程,记录其执行需要的开销。但是对于重写过程中的候选查询,很难直接确定其性能。因此,本研究额外设立了候选重写性能评估器来评估候选查询的性能,确定重写结果是否比原来更优。

针对查询性能评估任务,可以使用数据库自带的EXPLAIN成本估计作为执行效率的预测,或者由数据库实际执行该SQL语句来测量真实的查询延迟。测量真实延迟的方法尤其适用于具有固定周期(每月/每周/每日等)的重复查询的工作负载。在本框架中,默认使用EXPLAIN方法来评估候选重写的性能表现。这样可以复用语法检测的结果,降低整体时间与资源开销。

候选重写评估模块会返回一个布尔型的值 True 或 False,表示重写结果的有效性。如果一个方法在单条查询上重写后,重写前后的执行结果完全相同,且没有发生性能退化,则定义该重写是正确的有效重写,即重写成功。

2.3.3 自修正反馈

如果重写失败,系统调用自主修正机制,把错误的

重写结果发回大语言模型再次重写,并作为负面案例供参考。这个过程会循环最多 η 次,默认取 $\eta=3$ 。如果超过 η 次还没有重写成功,那么会直接输出原始查询,保证结果的正确性。新查询生成模块会在重写过程中记录规则的使用效果,并反馈回知识库中。

本框架通过知识库增强模块实现多源知识融合,设计动态提示工程,利用已有的数据库元信息、重写规则等多类信息引导大语言模型生成准确的优化方案。在重写过程中还会不断吸收信息生成新重写规则,持续提高查询重写效果。并建立语法-语义-形式化验证、候选重写评估与自主修正机制保障生成结果可靠性。

3 实验与分析

3.1 实验设置

本研究选取了 TPC-H、DSB^[21]、IMDB^[22] 3 组不同的数据集作为实验基准,涵盖了多个领域的复杂查询。考虑到 GPT-4 等大语言模型在训练过程中可能已经使用了本研究采用的公开数据集,在本实验中对数据集中的表名、列名进行了混淆,隐藏实际语义信息。

主要对比下面 5 种方法:

(1) Original: WookongDB 原有基于固定规则的查询重写方法。

(2) LLM Only: 朴素大语言模型方法,没有引入 RAG 技术辅助查询重写。

(3) LR (learned query rewrite system)^[14]: 基于蒙特卡洛树的最优查询搜索方法,基于机器学习小模型的 SOTA 方法。

(4) LLM-R2^[13]: 引入课程学习技术,基于大语言模型进行查询重写,但仅仅检索已有的重写规则,不支持规则合成。

(5) Wookong-QR: 基于本研究提出新技术设计的 SQL 查询智能重写系统。

为了评估云数据库 WookongDB 在不同 SQL 查询重写方法下的表现,本研究主要关注几个核心指标,即执行效率、重写开销与重写有效性,后续实验分别基于这些核心指标展开。

执行效率是本实验的核心评估指标,主要是执行时间,也称查询时延。通过记录执行时间的平均值、中位数与第 90 百分位,对执行效率进行全面的评估。其中平均执行时间和中位数执行时间体现了一般情况下的执行效率,而第 90 百分位数执行时间则考察各个方法在长尾问题下的表现情况。

评估重写开销是为了判断新的查询重写方法是否会拖慢原有数据库的整体性能。因为引入新方法必然

会造成额外开销,这就需要在执行时间与重写开销之间取得良好平衡。重写开销包含查询重写过程中需要消耗的时间、系统资源等各项开销。其中最重要也是本实验主要关注的是查询重写的时间开销,即重写时延。为了缓解波动,每条查询将在相同环境下执行 3 遍,取平均时间。

重写有效性的评估标准是查询重写方法的有效率。因为 WookongDB 原有查询重写方法基于固定规则,可以认为是完全正确且有效的,并作为其他方法的比较基准。即 Original 方法的重写有效率为 100%,而其他方法需要实验测定。

本研究在一个模拟的云环境中进行了实验,该环境由一个具备 RDMA(remote direct memory access)功能的 EDR(enhanced data rate)集群组成,包含 8 个节点:4 个计算节点、2 个内存节点、1 个存储节点与 1 个代理节点。此设置包括一个活跃的写入实例和一个活跃的读取实例。每个活跃的计算实例都有一个备份实例,在发生故障转移时接管客户端连接。每个活跃的计算实例配有一个内存节点用于上下文持久化。共享存储池托管在一个独立的节点上,确保数据访问和一致性。部署了一个代理节点,用于将客户端请求转发到计算实例,并且把 Wookong-QR 系统部署在代理节点上。集群中的每个节点都配备了一台搭载 Intel Xeon Gold 5220 CPU (18 核心 $\times$ 2 超线程,主频 2.20 GHz)、128 GB DRAM 内存以及 Mellanox ConnectX-5 InfiniBand 网络接口卡(最高支持 100 Gbit/s EDR 速率)的机器。由于代理节点部署有 Wookong-QR 系统,该节点设有额外的 GPU 硬件 Nvidia RTX 5880 Ada (48 GB) $\times$ 2。

3.2 执行效率评估实验

本实验对重写后的 SQL 查询的执行时间进行比较分析。为了防止有些过于复杂的查询拖慢实验进程,本实验参考相关工作设置了最大时间限制,取 300 s,实际执行时间超过此限制的查询将被截断。在本实验中,各方法在前期准备工作中都需要一定的时间与资源开销,例如 Wookong-QR 需要准备文档、初始化知识库等,这些少量开销将不被计入。

首先横向比较各个方法跨基准的执行时间,即对比 Original、LLM Only、LR、LLM-R2 以及本文 Wookong-QR 5 种方法在 TPC-H(见图 2)、DSB(见图 3)、IMDB 数据集(见图 4)上的执行时间平均值、中位值与 p90 值(单位:s)。

综合以上 3 组负载下的实验结果,可以看出:

(1) 整体来看, Wookong-QR 是全局最优的方法。Wookong-QR 在 TPC-H 负载下实现 37.64 s 平均执行时

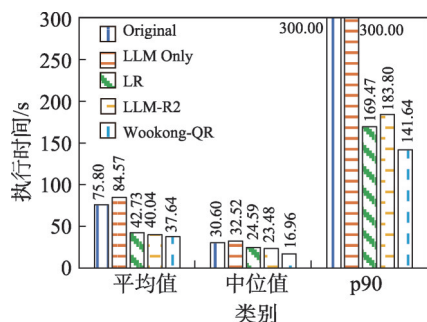


图2 TPC-H负载执行性能测试

Fig.2 Performance testing of TPC-H benchmark workloads

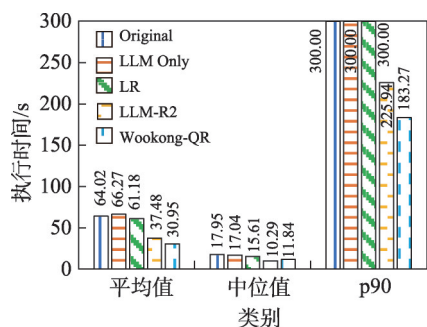


图3 DSB负载执行性能测试

Fig.3 Performance testing of DSB benchmark workloads

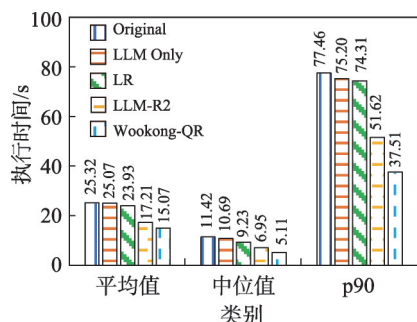


图4 IMDB负载执行性能测试

Fig.4 Performance testing of IMDB benchmark workloads

间,较次优的 LLM-R2 提升 5.9%,相比原始方法(Original)优化幅度达 50.3%。在 DSB 数据集上, Wookong-QR 以 30.95 s 平均时间保持领先,较 LLM-R2 提升 17.4%,较 LR 方法显著优化 49.4%。IMDB 场景中, Wookong-QR 的 15.07 s 平均时间达到最优,相比 LLM-R2 优化 12.4%,且 p90 值降低 27.3%。

(2)跨数据集比较, Wookong-QR 平均执行时间排序为 TPC-H(37.64 s)>DSB(30.95 s)>IMDB(15.07 s),这与各数据集查询复杂度相关。DSB 表现出最显著的长尾效应, Original 方法的 p90/中值比达到 16.7 倍, Wookong-QR 将此比例优化至 15.5 倍。IMDB 的中位时间

最优(5.11 s),反映其 OLTP(online transaction processing)型负载特征, Wookong-QR 在此场景实现 55.3%的中位时间优化。

(3) Wookong-QR 在控制查询延迟长尾方面表现突出。在 TPC-H 与 DSB 这两类主要由复杂查询组成的负载中, WookongDB 原始查询重写器(Original)和朴素大语言模型方法(LLM Only)都出现了 p90 为 300 s 超时的长尾现象, LR 方法在 DSB 负载中也出现了此现象。LLM-R2 以及 Wookong-QR 在所有负载中都表现良好,没有出现这样严重的超时情况。同为基于大语言模型的查询重写方法,在 TPC-H 负载中 Wookong-QR 的 p90 执行时间降至 141.64 s,相比 LLM-R2 降低 22.9%;DSB 场景下 p90 时间 183.27 s,相比 LLM-R2 降低 18.9%。值得注意的是,在 TPC-H 数据集的 p90 数据中, LLM-R2 (183.80 s)>LR(169.47 s)>Wookong-QR(141.64 s)。因为考虑到了 WookongDB 分层云原生架构等多源信息且生成了初始规则库之外的多样规则, Wookong-QR 改写的查询取得了最优的执行开销。LLM-R2 和 LR 只考虑已有的重写规则,没有发现新的规则,重写的优化效果有限。LR 优于 LLM-R2 是因为 LR 工作中已经针对 TPC-H 做出了额外优化,绝大多数用于 TPC-H 数据集的有效重写规则已经预置在 LR 中。

3.3 重写开销评估

本实验主要通过测量查询重写消耗的平均时间来评估重写开销,横向对比 LLM Only、LR、LLM-R2、Wookong-QR 4 种方法。考虑到 SQL 决策分类方法的消融实验,将 Wookong-QR 分为两个部分,包括无 SQL 分类和有 SQL 分类。平均查询重写时间实验结果如图 5 所示。

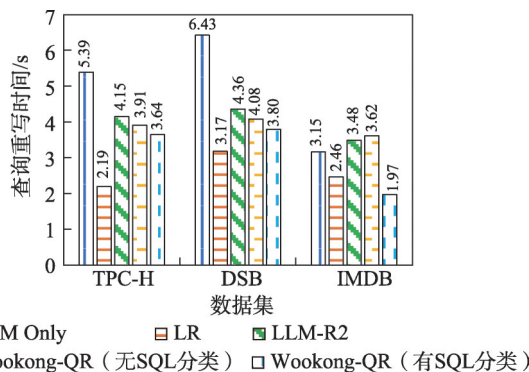


图5 查询重写开销测试

Fig.5 Testing overhead of query rewriting

查询重写时间和执行所需要的平均总时间如图 6 所示。

数据显示 Wookong-QR 及其 SQL 决策分类能有效提升重写效率,具体而言:

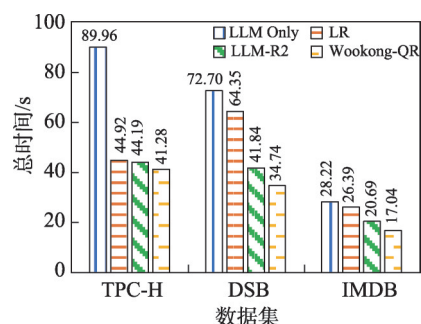


图6 查询重写与执行总开销测试

Fig.6 Testing total overhead of query rewriting and execution

(1) 重写效率较高。Wookong-QR(有SQL分类)在所有数据集上的平均时间(TPC-H: 3.64 s, DSB: 3.80 s, IMDB: 1.97 s)均低于除了LR以外的其他方法,包括LLM Only与LLM-R2。尽管在查询重写时间上略大于LR,但这是引入大语言模型和知识库等模块的必须开销。从上一节中可以看到,Wookong-QR比LR的执行时间有所缩短,整体效率仍然更高。

(2) 消融验证有效性。在IMDB数据集上,带有SQL分类的版本将重写时间压缩至1.97 s,较次优方案LR(2.46 s)降低19.9%,而LLM Only方法在该场景耗时高达3.15 s。对比Wookong-QR有无分类的两个版本,有分类相较无分类版本分别降低6.9%(TPC-H)、6.9%(DSB)和45.5%(IMDB),分类机制对复杂负载会起到一定优化效果。因为IMDB数据集基础型查询较多,SQL决策分类在该数据集上优势格外明显。这印证了SQL决策分类通过提前筛选复杂型查询,有效规避了大语言模型在基础型查询上的低效计算,从而系统性降低重写开销。

总成本与重写轮数呈线性关系。实验结果充分证明,将大语言模型与传统查询优化器相结合的混合架构,能够有效平衡创新性与可靠性。

4 总结

本文针对WookongDB中目前在云原生等复杂环境下查询重写效果不佳的现状,提出了一系列面向WookongDB的查询重写优化技术,包括基于大语言模型的查询重写框架、SQL查询分类决策机制、混合负载感知的查询特征提取方法、云环境下多源知识获取机制、查询语法-语义-形式化验证、候选重写评估、重写结果自主修正机制等,并在此基础上设计并实现了面向WookongDB的SQL查询智能重写系统Wookong-QR。它由重写预备、知识库增强与新查询生成3个模块组成。

为了评估Wookong-QR的重写效果与实际性能,本文做了一系列对比实验。实验表明,相比WookongDB

原有查询重写器、朴素大语言模型以及查询重写相关工作,Wookong-QR在TPC-H、DSB与IMDB负载下的查询响应时间普遍降低5%以上,并且通过SQL查询分类机制成功降低查询重写本身的开销,重写开销较分类之前降低6.9%以上。实验结果充分验证了本文技术方案及Wookong-QR系统的可行性与高效性。

参考文献:

- [1] 杜小勇, 卢卫, 张峰. 大数据管理系统的历史、现状与未来[J]. 软件学报, 2019, 30(1): 127-141.
DU X Y, LU W, ZHANG F. History, present, and future of big data management systems[J]. Journal of Software, 2019, 30(1): 127-141.
- [2] 杜小勇, 陈跃国, 范举, 等. 数据整理: 大数据治理的关键技术[J]. 大数据, 2019, 5(3): 13-22.
DU X Y, CHEN Y G, FAN J, et al. Data wrangling: a key technique of data governance[J]. Big Data Research, 2019, 5(3): 13-22.
- [3] LI F F. Cloud-native database systems at Alibaba: opportunities and challenges[J]. Proceedings of the VLDB Endowment, 2019, 12(12): 2263-2272.
- [4] LI G L, DONG H W, ZHANG C. Cloud databases: new techniques, challenges, and opportunities[J]. Proceedings of the VLDB Endowment, 2022, 15(12): 3758-3761.
- [5] CAO W, ZHANG Y Q, YANG X J, et al. PolarDB serverless: a cloud native database for disaggregated data centers[C]// Proceedings of the 2021 International Conference on Management of Data. New York: ACM, 2021: 2477-2489.
- [6] VERBITSKI A, GUPTA A, SAHA D, et al. Amazon Aurora: design considerations for high throughput cloud-native relational databases[C]// Proceedings of the 2017 ACM International Conference on Management of Data. New York: ACM, 2017: 1041-1052.
- [7] VERBITSKI A, GUPTA A, SAHA D, et al. Amazon Aurora: on avoiding distributed consensus for I/Os, commits, and membership changes[C]// Proceedings of the 2018 International Conference on Management of Data. New York: ACM, 2018: 789-796.
- [8] ANTONOPOULOS P, BUDOVSKI A, DIACONU C, et al. Socrates: the new SQL server in the cloud[C]// Proceedings of the 2019 International Conference on Management of Data. New York: ACM, 2019: 1743-1756.
- [9] WANG J Y, LI T L, SONG H Z, et al. PolarDB-IMCI: a cloud-native HTAP database system at Alibaba[J]. Proceedings of the ACM on Management of Data, 2023, 1(2): 1-25.
- [10] LI G, ZHOU X, CAO L. AI meets database: AI4DB and

- DB4AI[C]//Proceedings of the 2021 International Conference on Management of Data. New York: ACM, 2021: 2859-2866.
- [11] LI G L, ZHOU X H, ZHAO X Y. LLM for data management [J]. Proceedings of the VLDB Endowment, 2024, 17(12): 4213-4216.
- [12] LIU J, MOZAFARI B. Query rewriting via large language models[EB/OL]. [2025-04-16]. <https://arxiv.org/abs/2403.09060>.
- [13] LI Z, YUAN H T, WANG H M, et al. LLM-R²: a large language model enhanced rule-based rewrite system for boosting query efficiency[J]. Proceedings of the VLDB Endowment, 2024, 18(1): 53-65.
- [14] ZHOU X H, LI G L, WU J M, et al. A learned query rewrite system[J]. Proceedings of the VLDB Endowment, 2023, 16(12): 4110-4113.
- [15] ZHOU F, XUE S Q, QI D R, et al. DB-GPT-Hub: towards open benchmarking text-to-SQL empowered by large language models[EB/OL]. [2025-04-16]. <https://arxiv.org/abs/2406.11434>.
- [16] ZHOU X H, LI G L, SUN Z Y, et al. D-Bot: database diagnosis system using large language models[EB/OL]. [2025-04-16]. <https://arxiv.org/abs/2312.01454>.
- [17] SUN Z Y, ZHOU X H, LI G L. R-Bot: an LLM-based query rewrite system[EB/OL]. [2025-04-16]. <https://arxiv.org/abs/2412.01661>.
- [18] ZHANG X Y, WU H, LI Y, et al. Towards dynamic and safe configuration tuning for cloud databases[C]//Proceedings of the 2022 International Conference on Management of Data. New York: ACM, 2022: 631-645.
- [19] WANG Z G, ZHOU Z, YANG Y C, et al. WeTune: automatic discovery and verification of query rewrite rules[C]//Proceedings of the 2022 International Conference on Management of Data. New York: ACM, 2022: 94-107.
- [20] DONG R, LIU J, ZHU Y X, et al. SlabCity: whole-query optimization using program synthesis[J]. Proceedings of the VLDB Endowment, 2023, 16(11): 3151-3164.
- [21] DING B L, CHAUDHURI S, GEHRKE J, et al. DSB: a decision support benchmark for workload-driven and traditional database systems[J]. Proceedings of the VLDB Endowment, 2021, 14(13): 3376-3388.
- [22] LEIS V, GUBICHEV A, MIRCHEV A, et al. How good are query optimizers, really?[J]. Proceedings of the VLDB Endowment, 2015, 9(3): 204-215.



徐言诗(1999—),男,硕士,主要研究方向为云原生数据库。

XU Yanshi, born in 1999, M.S. His research interest is cloud native databases.



张心怡(1997—),女,博士,讲师,主要研究方向为自治数据库系统。

ZHANG Xinyi, born in 1997, Ph.D., lecturer. Her research interest is autonomous database systems.



卢卫(1981—),男,博士,教授,博士生导师,主要研究方向为数据库系统。

LU Wei, born in 1981, Ph.D., professor, Ph.D. supervisor. His research interest is database systems.



杜小勇(1963—),男,博士,教授,博士生导师,主要研究方向为数据库系统。

DU Xiaoyong, born in 1963, Ph.D., professor, Ph.D. supervisor. His research interest is database systems.